Théorie des Graphes – Rapport du Projet

Sujet 1 : Flot Maximal dans un Réseau

Tom VOCCIA & Ngoc Nhu Y GIAN

27 mars 2026

Table des matières

1 Introduction **2**

2 Architecture et Stratégies **2**
2.1 Langage et Environnement : 2
2.2 Structure de Données : 2
2.3 Gestion du Graphe Résiduel 2
2.4 Modularité : 3

3 Implémentation Algorithmique **3**
3.1 Algorithme d'Edmonds-Karp 3
3.2 Théorème de Menger (Approche Combinatoire) 3
3.3 Analyse spectrale (valeur de Fiedler) 3

4 Analyse Expérimentale **4**
4.1 Protocole de Test 4
4.2 Résultats (Benchmarks) 4
4.3 Discussion des Résultats 5

5 Manuel d'Installation et Utilisation **5**
5.1 Prérequis 5
5.2 Installation (environnement virtuel et dépendances) 5
5.3 Lancement 5
5.4 Utilisation de l'interface graphique 5
5.5 Format des fichiers de graphe (`data/*.txt`) 6
5.6 Fichiers de sortie 6
5.7 Nettoyage 6

1 Introduction

Le projet du cours de Théorie des Graphes porte sur la résolution du problème du *Flot Maximal dans un réseau*. L'objectif fondamental est de modéliser un réseau de transport (routes, canalisations, données) contraint par des capacités limitées, afin de déterminer la quantité optimale de flux pouvant transiter d'une *Source* vers un *Puits*.

Pour répondre à cette problématique, nous avons développé une solution logicielle modulaire en langage *Python*. Notre approche repose sur l'implémentation de l'algorithme d'*Edmonds-Karp*, une méthode algorithmique robuste utilisant *un parcours en largeur (BFS)* pour garantir la découverte du chemin le plus court dans le graphe résiduel.

Le projet se divise en deux parties :

- **La génération de données :** Une interface graphique permettant de créer des scénarios de test variés et aléatoires.
- **La résolution et l'analyse :** Le calcul du flot maximal, étendu à l'application du Théorème de Menger (chemins disjoints) et à une analyse spectrale de la connectivité du réseau.

2 Architecture et Stratégies

2.1 Langage et Environnement :

Le projet est développé en *Python 3*, choisi pour sa rapidité de prototypage et ses bibliothèques scientifiques (`NumPy`, `PyVis`) et une librairie graphique (`tkinter`). Afin de garantir la portabilité, un environnement virtuel (`venv`) est utilisé pour isoler les dépendances.

2.2 Structure de Données :

Afin d'optimiser l'efficacité spatiale et temporelle, nous avons privilégié une représentation par liste d'adjacence (via des dictionnaires Python) plutôt qu'une matrice d'adjacence dont la complexité spatiale en $O(V^2)$ est prohibitive pour les grands graphes.

- **Implémentation :** `self.adj[u][v] = [capacité, flux]`
- **Complexité :** Cette structure offre un accès en temps constant amorti $O(1)$ aux successeurs d'un sommet, accélérant ainsi le parcours BFS.
- **Avantages :** Elle est particulièrement adaptée aux graphes creux (*sparse graphs*) et permet de centraliser les données statiques (capacité) et dynamiques (flux courant) dans une unique structure, simplifiant la mise à jour du graphe résiduel.

2.3 Gestion du Graphe Résiduel

L'algorithme de flot maximal reposant sur la capacité à "annuler" un flux déjà affecté (principe du graphe résiduel), nous avons opté pour une instanciation systématique des arcs inverses dès la création du graphe. Pour tout arc $u \to v$ ajouté, l'arc retour $v \to u$ est immédiatement créé avec une capacité nulle.

```
# Creation simultanee de l'arc et de son inverse
self.adj[u][v] = [capacity, 0]  # Arc Aller (Capacite reelle)
self.adj[v][u] = [0, 0]         # Arc Retour (Capacite virtuelle 0)
```

Cette stratégie d'anticipation permet d'uniformiser le traitement des arcs lors du parcours en largeur (BFS). L'algorithme n'a pas besoin de distinguer un arc "réel" d'un arc "fantôme" : il se contente de vérifier si la capacité résiduelle est positive, ce qui simplifie considérablement l'implémentation de la fonction `edmonds_karp`.

2.4 Modularité :

Pour réaliser ce projet, nous avons opté pour une architecture modulaire stricte. Le projet est divisé en six fichiers distincts, chacun assumant une responsabilité unique au sein du système. Cette séparation garantit une meilleure lisibilité et facilite le travail collaboratif.

- `src/affichage.py` **(IHM) :** Gère l'interface graphique (*Tkinter*) pour la configuration des paramètres et le lancement des opérations.
- `src/graph_generator.py` **(Génération) :** Assure la création procédurale de graphes aléatoires valides et le respect des contraintes (pas de multi-arêtes).
- `src/io_manager.py` **(Persistance) :** S'occupe de la lecture (*parsing*) des fichiers d'entrée standardisés et de l'écriture des résultats sur le disque.
- `src/graph.py` **(Données) :** Définit la classe `Graph` (liste d'adjacence), structure en mémoire vive utilisée par tous les algorithmes.
- `src/algorithm.py` **(Calcul) :** Contient la logique mathématique pure : BFS, Edmonds-Karp, Théorème de Menger et Analyse Spectrale.
- `src/visualizer.py` **(Rendu) :** Transforme les résultats numériques en une visualisation web interactive via la librairie *PyVis*.

Cette organisation permet une séparation nette entre la création du problème (Génération), sa résolution (Algorithme) et sa présentation (Visualisation).

3 Implémentation Algorithmique

3.1 Algorithme d'Edmonds-Karp

L'implémentation de la fonction `Edmonds-Karp` repose sur la combinaison de la méthode de *Ford-Fulkerson* et d'un algorithme de recherche de chemin spécifique : le *BFS (Breadth-First Search)* :

- **1. Le rôle du BFS (Parcours en Largeur) :** Il permet d'explorer le graphe niveau par niveau à partir de la source. Son objectif est de déterminer le chemin le plus court en nombre d'arêtes vers le puits. Cela garantit une complexité temporelle maîtrisée pour l'algorithme global..
- **2. Le rôle de Ford-Fulkerson :** C'est la structure générale de résolution. Elle itère tant qu'il existe un chemin valide dans le graphe résiduel, c'est-à-dire un chemin composé uniquement d'arcs dont la capacité n'est pas saturée (où le flux actuel est inférieur à la capacité maximale).

3.2 Théorème de Menger (Approche Combinatoire)

L'implémentation de la fonction `theorem_menger` exploite le théorème de Menger dans sa version arête-disjointe. L'objectif est de trouver le nombre maximal de chemins reliant la source au puits sans jamais emprunter deux fois la même arête.

Pour ce faire, nous transformons le problème en un calcul de flot : en fixant la capacité de chaque arc à 1, le flot maximal obtenu correspond exactement au nombre de ces chemins indépendants.

Théoriquement, ce résultat illustre la dualité Max-Flow Min-Cut : le nombre maximum de chemins arête-disjoints est égal à la capacité de la coupe minimum (c'est-à-dire le nombre minimal d'arêtes à supprimer pour rendre le puits inaccessible depuis la source).

3.3 Analyse spectrale (valeur de Fiedler)

L'implémentation de la fonction `analyse_spectrale` garantit le calcul rigoureux de la Valeur de Fiedler (λ_2) pour un graphe non orienté $G = (V, E)$ de taille $n = |V|$.

1. Préparation et Indexation La première étape cruciale est l'établissement d'une base ordonnée pour la représentation matricielle.

- **Ordonnancement des sommets :** L'opération `vertices = sorted(graph.get_all_vertices())` impose un ordre arbitraire mais **fixe** aux sommets. Cet ordre est essentiel pour que l'indexation matricielle soit déterministe.
- **Mappage :** Le dictionnaire `mapping` crée un isomorphisme $f : V \to \{0, \ldots, n-1\}$, associant chaque nom de sommet $v \in V$ à son indice matriciel $i \in \{0, \ldots, n-1\}$. Cela permet de garantir que l'élément $L_{i,j}$ correspond exactement à la relation entre les sommets $f^{-1}(i)$ et $f^{-1}(j)$.

2. Construction de la Matrice Laplacienne ($L = \mathbf{D} - A$) Le code construit la matrice Laplacienne L sans nécessiter l'instanciation séparée des matrices $\mathbf{D}$ et A, optimisant l'efficacité.

$$L_{i,j} = \begin{cases} \deg(v_i) & \text{si } i = j \text{ (diagonale)} \\ -1 & \text{si } (v_i, v_j) \in E \text{ (arête)} \\ 0 & \text{sinon} \end{cases}$$

- **Hypothèse de Symétrie :** Le code suppose que le graphe est traité comme **non orienté**. Bien que l'itération `for v in graph.get_neighbors(u)` ne parcoure que les arêtes sortantes de u, la structure de données sous-jacente (`graph.adj[u][v]`) et l'objectif de l'analyse spectrale Laplacienne impliquent que l'arête (u, v) doit être traitée comme l'arête non orientée $\{u, v\}$. Pour les graphes non orientés, L est symétrique ($L_{i,j} = L_{j,i}$).
- **Éléments Hors Diagonale ($-A$) :** L'affectation `L[i][j] = -1` insère l'élément $-\mathbf{A}_{i,j}$ pour toute arête détectée. L'hypothèse de non-orientation assure que lorsque le sommet v sera traité dans la boucle externe, l'élément $L[j][i]$ sera également mis à -1, préservant la symétrie.
- **Éléments Diagonaux (D) :** La variable `d_u` accumule le degré $\deg(u)$ du sommet u. L'instruction `L[mapping[u]][mapping[u]] = d_u` place le degré total sur la diagonale $L_{i,i}$. Ceci complète la construction de $L = \mathbf{D} - A$.

3. Calcul des Valeurs Propres et Identification

- **Choix de la Fonction NumPy :** La fonction `np.linalg.eigvalsh(L)` est utilisée. Le suffixe `-h` indique l'utilisation d'algorithmes optimisés pour les matrices **Hermitiennes** (symétriques dans le cas réel), garantissant une meilleure performance et une plus grande **stabilité numérique** pour les matrices Laplaciennes réelles.
- **Tri :** L'appel `vals.sort()` trie le spectre $\Lambda(L)$ par ordre croissant.
- **Valeur de Fiedler :** Par définition, la Valeur de Fiedler est λ_2. Le spectre trié commence à l'indice 0 ($\lambda_1 = 0$), l'indice **[1]** correspond donc rigoureusement à λ_2. L'expression `return vals[1] if len(vals) > 1 else 0` retourne λ_2 ou 0 si le graphe est trop petit.

Ce processus garantit que la valeur retournée est bien la connectivité algébrique, un indicateur direct de la cohésion du réseau étudié.

4 Analyse Expérimentale

4.1 Protocole de Test

Les tests ont été réalisés sur un processeur standard (Architecture x86_64, Linux Arch). Les graphes sont générés aléatoirement avec une densité variable via le script `generate_tests.py`.

4.2 Résultats (Benchmarks)

Le tableau suivant résume les performances obtenues sur différents ordres de grandeur :

Taille (N)	Arêtes (E)	Temps (s)	Flot Max	Menger (k)
10 (Petit)	≈ 20	0.0002	13	2
500 (Moyen)	$\approx 50\,000$	1.98	941	95
1000 (Grand)	$\approx 200\,000$	19.30	2119	202
5000 (Stress)	$\approx 5 \cdot 10^6$	> 5 min	-	-

TABLE 1 – Performances de l'algorithme Edmonds-Karp

4.3 Discussion des Résultats

1. **Efficacité :** L'algorithme traite un graphe de 1000 sommets et 200 000 arcs en moins de 20 secondes. Le passage à 5000 sommets entraîne une explosion combinatoire prévisible due à la complexité en $O(VE^2)$, confirmant les limites d'Edmonds-Karp pour les très grands réseaux denses.
2. **Robustesse (Menger vs Flot) :** Sur le graphe de 1000 sommets, nous trouvons un flot de 2119 mais seulement 202 chemins disjoints. Cela illustre la différence entre *débit* (capacité des tuyaux) et *fiabilité* (redondance des routes).
3. **Spectre :** La valeur de Fiedler obtenue pour le graphe de 1000 nœuds (≈ 160.8) confirme une très forte connectivité algébrique, typique des graphes aléatoires denses.

5 Manuel d'Installation et Utilisation

Le projet s'exécute localement à partir de l'archive fournie. L'exécution lance une interface graphique (Tkinter) permettant soit de **générer** un graphe, soit de **charger** un graphe existant.

5.1 Prérequis

— **Python 3** (recommandé : 3.8+).
— **Make**.
— **Tkinter** (interface graphique) : généralement inclus avec Python, mais sur certaines distributions Linux il faut installer le paquet système (ex. `python3-tk`).
— Accès à `pip` et, si nécessaire, aux outils de compilation pour installer certaines dépendances Python (cas typique sous Linux).

5.2 Installation (environnement virtuel et dépendances)

Depuis le dossier racine du projet (celui qui contient le `Makefile`) :

```
make install
```

Cette commande :
— crée un environnement virtuel local `./venv/` (si absent) ;
— installe automatiquement les dépendances listées dans `requirements.txt`.

5.3 Lancement

Pour exécuter le programme (et ouvrir l'interface graphique) :

```
make run
```

5.4 Utilisation de l'interface graphique

Au lancement, une fenêtre **Générateur de Graphe** s'ouvre. Deux modes sont disponibles :

1) Générer un graphe

- Renseigner :
  - **Nombre de Sommets**
  - **Nombre d'Arêtes**
  - **Capacité Maximale** (borne supérieure des capacités tirées aléatoirement)
  - **Source** et **Puits** (identifiants entiers des sommets)
  - **Nom du fichier (data/)** : le fichier est enregistré dans le dossier `data/`. L'extension `.txt` est ajoutée automatiquement si nécessaire.
- Cliquer sur **Générer le Graphe**.

2) Charger un graphe existant

- Cocher **Utiliser un fichier existant**.
- Saisir le **Nom du fichier (data/)** correspondant à un fichier déjà présent dans `data/` (extension `.txt` optionnelle).
- Renseigner **Source** et **Puits**.
- Cliquer sur **Charger le Graphe**.

Une fois validé, le solveur s'exécute automatiquement et affiche les résultats dans le terminal : flot maximal (Edmonds–Karp), nombre de chemins arête-disjoints (Menger) et valeur de Fiedler (analyse spectrale).

5.5 Format des fichiers de graphe (`data/*.txt`)

Chaque ligne décrit un arc orienté sous la forme :

$$u \;\; v \;\; cap$$

où u et v sont des entiers (sommets) et cap une capacité entière ≥ 0. Les lignes vides et les commentaires commençant par `#` sont ignorés.

5.6 Fichiers de sortie

- `data/output.txt` : solution de flot (une ligne par arc réel) au format `u v flow`.
- `initial_graph.html` : visualisation interactive du réseau initial (PyVis) générée dans le dossier racine du projet avec un flot de 1 dans chaques arêtes.
- `flow_solution.html` : visualisation interactive du réseau optimisé (PyVis) générée dans le dossier racine du projet (ne montre que les arêtes dont le flow est strictement supérieur à 0).

5.7 Nettoyage

Pour supprimer l'environnement virtuel et les caches Python :

```
make clean
```

Bibliographie

1 B. Korte et J. Vygen, *Combinatorial Optimization*, Springer, Chapitre 8. Algorithme d'Edmonds-Karp et Théorème de Menger.

2 M. Rigo, *Advanced Graph Theory and Combinatorics*, Wiley-ISTE, 2016. Porte d'ouverture vers des concepts plus avancé comme l'analyse spectrale sur base de la matrice Laplacienne.

3 M. Fiedler, *Algebraic connectivity of graphs*, Czechoslovak Math. J, 23 :298-305 (1973). Pour la valeur de Fiedler et son utilité pour mesurer la robustesse d'un graphe.